B.TECH V SEMESTER

Selected AIML Subject Syllabus

DVR & Dr. HS MIC College of Technology (Autonomous)

B.Tech – Artificial Intelligence & Machine Learning

Extracted from the supplied curriculum PDF, pages 113–139

Included: Deep Learning, Computer Networks, Operating Systems, Full Stack Development-2, User Interface Design using Flutter, Deep Learning Lab, and Operating Systems & Computer Networks Lab.

Important: The official syllabus lists Operating Systems & Computer Networks LAB as one combined course (23AM5L11). The source does not define separate OS Lab and CN Lab syllabi, so this document keeps the official combined lab structure.

23AM5T01 — DEEP LEARNING

Course Type: PC L–T–P–C: 3–0–0–3 Source pages: 113–114

Course Objectives

1. Provide foundational knowledge in linear algebra and vectorization techniques essential for deep learning.

2. Introduce and explain the architecture and training of neural networks.

3. Explore advanced optimization and regularization techniques to improve training.

4. Develop an understanding of recurrent and convolutional neural networks and generative models.

5. Introduce recent trends in generative AI and transformer-based architectures with real-world applications.

Course Outcomes

1. Understand and apply linear algebra concepts in the context of deep learning.

2. Analyze and implement perceptron models and basic learning algorithms.

3. Design and train deep feedforward networks using various optimization techniques.

4. Apply and evaluate advanced neural network architectures like CNNs, RNNs, and LSTMs.

5. Identify applications of generative AI in vision, NLP, and speech, and understand agentic AI.

Syllabus

UNIT-I: Basics of linear algebra for Deep Learning

Basics of linear algebra for Deep Learning: Matrices, types, Inverse, Eigen values and vectors, Principal component analysis, Image and text vectorization, Biological Neuron, Idea of computational units, McCulloch–Pitts unit and Thresholding logic, Linear Perceptron, Perceptron Learning Algorithm, Linear separability, Convergence theorem for Perceptron Learning Algorithm.

UNIT-II: Feed forward Networks

Feed forward Networks-Multilayer Perceptron, Gradient Descent, Backpropagation, Empirical Risk Minimization, regularization. Deep Neural Networks: Difficulty of training deep neural networks, Greedy layer wise training.

UNIT-III: Better Training of Neural Networks

Better Training of Neural Networks-Newer optimization methods for neural networks (Adagrad, adadelta, rmsprop, adam, NAG), second order methods for training, Saddle point problem in neural networks, Regularization methods (dropout, drop connect, batch normalization).

UNIT-IV: Recurrent Neural Networks

Recurrent Neural Networks- Back propagation through time, Long Short Term Memory, Gated Recurrent Units, Bidirectional LSTMs, Bidirectional RNNs. Convolutional Neural Networks: Convolution operation, types of convolutions, LeNet, AlexNet. Generative models: Restrictive Boltzmann Machines (RBMs), Introduction to MCMC and Gibbs Sampling, gradient computations in RBMs, Deep Boltzmann.

UNIT-V: Recent trends

Recent trends-Variational Auto encoders, Transformers, GPT Applications: Vision, NLP, Speech, Introduction to Generative AI-Types,Applications,Agentic AI-Applications.

23AM5T02 — COMPUTER NETWORKS

Course Type: PC L–T–P–C: 3–0–0–3 Source pages: 115–117

Course Objectives

1. To provide insight about networks, topologies, and the key concepts.

2. To gain comprehensive knowledge about the layered communication architectures (OSI and TCP/IP) and its functionalities.

3. To understand the principles, key protocols, design issues, and significance of each layers in ISO and TCP/IP.

4. To know the basic concepts of network services and various network applications.

Course Outcomes

1. Demonstrate different network models for networking links OSI, TCP/IP, B-ISDN, N-BISDN and get knowledge about various communication techniques, methods and protocol standards.

2. Discuss different transmission media and different switching networks.

3. Analyze data link layer services, functions and protocols like HDLC and PPP.

4. Compare and Classify medium access control protocols like ALOHA, CSMA, CSMA/CD, CSMA/CA, Polling, Token passing, FDMA, TDMA, CDMA protocols.

5. Determine application layer services and client server protocols working with the client server paradigms like WWW, HTTP, FTP, e-mail and SNMP etc.

Syllabus

UNIT-I: Introduction

Introduction: Network Types, LAN, MAN, WAN, Network Topologies Reference models-The OSI Reference Model- the TCP/IP Reference Model - A Comparison of the OSI and TCP/IP Reference Models, OSI Vs TCP/IP. Physical Layer –Introduction to Guided Media- Twisted-pair cable, Coaxial cable and Fiber optic cable and introduction about unguided media.

UNIT-II: Data link layer

Data link layer: Design issues, Framing: fixed size framing, variable size framing, flow control, error control, error detection and correction codes, CRC, Checksum: idea, one’s complement internet checksum, services provided to Network Layer, Elementary Data Link Layer protocols: simplex protocol, Simplex stop and wait, Simplex protocol for Noisy Channel. Sliding window protocol: One bit, Go back N, Selective repeat-Stop and wait protocol, Data link layer in HDLC, Point to point protocol (PPP).

UNIT-III: Media Access Control & The Network Layer Design Issues

Media Access Control: Random Access: ALOHA, Carrier sense multiple access (CSMA), CSMA with Collision Detection, CSMA with Collision Avoidance, Controlled Access: Reservation, Polling, Token Passing, Channelization: frequency division multiple Access(FDMA), time division multiple access(TDMA), code division multiple access(CDMA). The Network Layer Design Issues – Store and Forward Packet Switching-Services Provided to the Transport layer-Implementation of Connectionless Service-Implementation of Connection Oriented Service- Comparison of Virtual Circuit and Datagram Networks.

UNIT-IV: Routing Algorithms & Internet Working

Routing Algorithms-The Optimality principle- Shortest path, Flooding, Distance vector, Link state, Hierarchical, Congestion Control algorithms-General principles of congestion control, Congestion prevention polices, Approaches to Congestion Control-Traffic Aware Routing-Admission Control-Traffic Throttling-Load Shedding. Traffic Control Algorithm-Leaky bucket & Token bucket. Internet Working: How networks differ- How networks can be connected- Tunneling, internetwork routing, Fragmentation, network layer in the internet – IP protocols-IP Version 4 protocol-IPV4 Header Format, IP addresses, Class full Addressing, CIDR, Subnets-IP Version 6-The main IPV6 header, Transition from IPV4 to IPV6, Comparison of IPV4 & IPV6.

UNIT-V: The Transport Layer & Application Layer

The Transport Layer: Transport layer protocols: Introduction-services- port number-User data gram protocol-User datagram-UDP services-UDP applications-Transmission control protocol: TCP services- TCP features- Segment- A TCP connection- windows in TCP- flow control-Error control, Congestion control in TCP. Application Layer – World Wide Web: HTTP, Electronic mail-Architecture- web based mail-email security- TELNET-local versus remote Logging-Domain Name System.

23AM5T03 — OPERATING SYSTEMS

Course Type: PC L–T–P–C: 3–0–0–3 Source pages: 118–119

Course Objectives

1. Understand the basic concepts and principles of operating systems, including process management, memory management, file systems, and Protection.

2. Make use of process scheduling algorithms and synchronization techniques to achieve better performance of a computer system.

3. Illustrate different conditions for deadlock and their possible solutions.

Course Outcomes

1. Understand the role and structure of an operating system.

2. Apply process and thread concepts.

3. Analyze CPU scheduling algorithms.

4. Manage memory effectively.

5. Understand and implement file system concepts.

Syllabus

UNIT-I: Operating Systems Overview

Introduction, Operating system functions, Operating systems operations, Computing environments, Free and Open-Source Operating Systems System Structures: Operating System Services, User and Operating-System Interface, system calls, Types of System Calls, system programs, Operating system Design and Implementation, operating system structure, Building and Booting an Operating System, Operating system debugging.

UNIT-II: Processes

Process Concept, Process scheduling, Operations on processes, Inter-process communication. Threads and Concurrency: Multithreading models, Thread libraries, Threading issues. CPU Scheduling: Basic concepts, Scheduling criteria, Scheduling algorithms, Multiple processor scheduling.

UNIT-III: Synchronization Tools

The Critical Section Problem, Peterson’s Solution, Mutex Locks, semaphores, Monitors, Classic problems of Synchronization. Deadlocks: system Model, Deadlock characterization, Methods for handling Deadlocks, Deadlock prevention, Deadlock avoidance, Deadlock detection, Recovery from Deadlock.

UNIT-IV: Memory-Management Strategies

Introduction, Contiguous memory allocation, Paging, Structure of the Page Table, Swapping. Virtual Memory Management: Introduction, Demand paging, Copy-on-write, Page replacement Allocation of frames, Thrashing Storage Management: Overview of Mass Storage Structure, HDD Scheduling.

UNIT-V: File System

File System Interface: File concept, Access methods, Directory Structure; File system Implementation: File-system structure, File-system Operations, Directory implementation, Allocation method, Free space management; File-System Internals: File-System Mounting, Partitions and Mounting, File Sharing. Protection: Goals of protection, Principles of protection, Protection Rings, Domain of protection, Access matrix.

23AM5S12 — FULL STACK DEVELOPMENT-2

Course Type: SC L–T–P–C: 0–0–4–2 Source pages: 135–137

Course Objectives

1. To provide comprehensive knowledge of front-end and back-end technologies used in modern web development.

2. To enable students to design, develop, and deploy scalable and responsive web applications using advanced tools, frameworks, and programming languages.

3. To impart hands-on experience with RESTful APIs, database integration, and authentication mechanisms.

4. To cultivate problem-solving skills and coding practices through project-based learning and realtime case studies.

Course Outcomes

1. Design and implement RESTful APIs using ExpressJS, incorporating routing, HTTP methods (GET, POST, PUT, DELETE), and middleware for request handling and validation.

2. Utilize templating engines like EJS to render dynamic HTML pages and handle form data submission and processing in ExpressJS applications.

3. Implement session management using cookies and sessions in ExpressJS, and develop user authentication mechanisms to secure web applications.

4. Connect ExpressJS applications to MongoDB using Mongoose, and perform CRUD operations to manage data effectively.

5. Develop ReactJS applications by rendering HTML elements, writing JSX syntax, and creating both functional and class-based components for dynamic user interfaces.

List of Experiments

1. Express JS – Routing, HTTP Methods, Middleware. a) Define routes, handling routes, route parameters, query parameters and URL building. b) Accept, retrieve and delete a specified resource using HTTP methods. c) Show middleware working.

2. Express JS – Templating, Form Data. a) Use a templating engine. b) Work with form data.

3. Express JS – Cookies, Sessions, Authentication. a) Session management using cookies and sessions. b) User authentication.

4. ExpressJS – Database, RESTful APIs. a) Connect MongoDB using Mongoose and perform CRUD. b) Develop a single page application using RESTful APIs.

5. ReactJS – Render HTML, JSX, Components – function & Class. a) Render HTML. b) Write markup with JSX. c) Create and nest functional and class components.

6. ReactJS – Props and States, Styles, Respond to Events. a) Work with props and states. b) Add CSS and Sass styling. c) Respond to events.

7. ReactJS – Conditional Rendering, Rendering Lists, React Forms. a) Conditional rendering. b) Rendering lists. c) Working with different form fields.

8. ReactJS – React Router, Updating the Screen. a) Routing with React Router. b) Updating the screen.

9. ReactJS – Hooks, Sharing data between Components. a) Understand hooks. b) Share data between components.

10. ReactJS Applications – To-do list and Quiz. a) Design a to-do list application.

11. MongoDB – Installation, Configuration, CRUD operations. a) Install MongoDB and configure ATLAS. b) CRUD queries using insert(), find(), update(), remove().

12. MongoDB – Databases, Collections and Records. a) Create and drop databases and collections. b) Work with records using find(), limit(), sort(), createIndex(), aggregate().

13-15. Augmented Programs: Any 2 must be completed. 13. To-do list using NodeJS and ExpressJS. 14. Quiz app using ReactJS. 15. Complete MongoDB certification from MongoDB University.

23AM5L13 — USER INTERFACE DESIGN USING FLUTTER

Course Type: ES L–T–P–C: 0–0–2–1 Source pages: 138–139

Course Objectives

1. Learns to implement Flutter widgets and layouts.

2. Understands responsive UI design and navigation in Flutter.

3. Knowledge on widgets and customizing widgets for specific UI elements and themes.

4. Understand to include animation apart from fetching data.

Course Outcomes

1. Develop responsive user interfaces in Flutter using core widgets, layouts, media-query breakpoints, and multi-screen navigation.

2. Apply state management, custom widgets, theming, forms, and validation techniques to build interactive and maintainable mobile app UIs.

3. Integrate RESTful data, animations, testing, and debugging to design polished, cross-platform Flutter applications.

4. Create custom widgets, apply themes and styles, and implement forms with validation to enhance user interaction and experience.

5. Integrate APIs for data handling, apply animations, and use testing and debugging tools to build robust and dynamic Flutter applications.

List of Experiments

1. Install Flutter and Dart SDK; write a simple Dart program for language basics.

2. Explore Flutter widgets such as Text, Image and Container; implement layouts using Row, Column and Stack.

3. Design a responsive UI for different screen sizes; implement media queries and breakpoints.

4. Set up navigation using Navigator; implement named routes.

5. Learn stateful and stateless widgets; use setState and Provider for state management.

6. Create custom widgets; apply themes and custom styles.

7. Design forms; implement validation and error handling.

8. Add animations; experiment with fade, slide and other animations.

9. Fetch data from a REST API and display it in the UI.

10. Write unit tests for UI components; use Flutter debugging tools.

23AM5L10 — DEEP LEARNING LAB

Course Type: PC L–T–P–C: 0–0–3–1.5 Source pages: 131–132

Course Objectives

1. Provide hands-on experience in building and training neural networks for classification and prediction tasks.

2. Familiarize students with convolution neural networks for image-based applications.

3. Introduce transfer learning using pre-trained models such as VGG16.

4. Apply text encoding techniques including one-hot encoding and word embeddings.

5. Design and evaluate recurrent neural networks for sequence modeling.

6. Use real-time image and video processing with object detection algorithms like YOLO.

Course Outcomes

1. Implement deep neural networks to solve real world problems.

2. Choose appropriate pre-trained model to solve real time problem.

3. Interpret the results of two different deep learning models.

4. Use one-hot encoding and word embeddings to preprocess textual data.

5. Use VGG16, YOLO and RNNs for image/video detection and sequence modeling.

List of Experiments

1. Implement multi-layer perceptron algorithm for MNIST handwritten digit classification.

2. Design a neural network for binary movie-review classification using IMDB.

3. Design a neural network for multi-class news-wire classification using Reuters.

4. Design a neural network for house-price prediction using Boston Housing.

5. Build a CNN for MNIST handwritten digit classification.

6. Build a CNN for dogs-and-cats image classification.

7. Use a pre-trained VGG16 model for image classification.

8. Implement one-hot encoding of words or characters.

9. Implement word embeddings for the IMDB dataset.

10. Implement video and image processing using YOLO algorithms.

11. Implement an RNN for next-word prediction.

23AM5L11 — OPERATING SYSTEMS & COMPUTER NETWORKS LAB

Course Type: PC L–T–P–C: 0–0–3–1.5 Source pages: 133–134

Course Objectives

1. Develop concepts of process and memory management techniques.

2. Develop concepts of process and memory management techniques.

3. Impart knowledge on developing shell scripts.

Course Outcomes

1. Implement CPU and disk scheduling algorithms.

2. Demonstrate memory management techniques.

3. Demonstrate algorithms for Deadlock Detection and prevention.

4. Develop shell scripts for shell programming.

5. Implement network-layer protocols and routing algorithms such as Sliding Window, Dijkstra’s algorithm and Distance Vector Routing.

List of Experiments

1. Simulate CPU scheduling: FCFS, SJF, Priority and Round Robin.

2. Simulate page replacement algorithms.

3. Implement LRU page replacement algorithm.

4. Illustrate Dead Lock Avoidance and Dead Lock Detection algorithms.

5. Illustrate UNIX commands and Vi editor.

6. Shell programs: check even/odd, factorial of first n natural numbers, and count lines and words in a file.

7. Implement data-link-layer framing methods: character stuffing and bit stuffing.

8. Implement Hamming Code generation for error detection and correction.

9. Implement CRC 12, CRC 16 and CRC CCIP on a character dataset.

10. Implement Sliding Window protocol for Selective Repeat.

11. Implement Dijkstra’s algorithm for shortest path.

12. Implement Distance Vector Routing and obtain routing tables at each node.